

C-Bounded Powersort's time complexity and correctness analysis

Théo Araújo Magalhães, Victor Almeida Campos

April 5, 2025

Abstract:

This paper presents a variation of Powersort, a stable mergesort algorithm. The original implementation of Powersort utilizes a $\log n + 1$ -sized stack of existing runs and sorts an array in a singular pass[3]. Our algorithm works by doing multiple passes over the array, keeping a limited stack of runs. We call this algorithm "C-Bounded Powersort" (placeholder). Moreover, we present an alternative and faster Node power algorithm and show that it gives the same guarantees as the one present in Munro & Wild's Powersort.

1 Introduction:

Sorting is a widely talked about problem in computer science, with a variety of novel ideas being tested in search of more efficient algorithms. Currently, a rather active field of research is that of sorting algorithms that adapt to their input. In our case, we are concerned with mergesort algorithms that adapt to the presence of existing sorted segments in the array, which is what we call a run. Mergesort algorithms that adapt to the presence of runs are often called "natural mergesorts"[3]. For further context and somewhat of a historiographical review, please check Munro & Wild's papers on Powersort and Multiway Powersort.[1][3]

1.1 Contributions:

We kind of need to do the experiments and run the stuff to talk about our contributions :) So this is a TO-DO!

1.2 Related works:

TO-DO!

2 Definitions:

Here, we define useful terms and ideas that will be frequently cited in our proofs section. Some of the definitions are not ours, some we tweaked to fit our purposes, and others we defined for better organization of the paper.

2.1 General definitions:

These definitions are related to generic natural mergesorts.

Runs: Contiguous regions of the input that are already in sorted order. Let $R_0, \dots, R_{r-1}$ be the runs in the input, L_i denotes the length of R_i . [3]

Run boundaries: Denoted by B_i , it is the boundary between the $(i - 1)$ st and i th run ($i=1, \dots, r - 1$). [1]

Node depth: d_i is the depth of leaf R_i in T , where depth is the number of edges on the path to the root. [1]

Run-adaptive mergesort and merge tree: Any stable run-adaptive mergesort can be described as a merge tree T : an (ordered rooted) tree with leaves $R_0, \dots, R_{r-1}$ (appearing in that order from left to right). Each internal node v of T corresponds to the result of merging its children. In binary merge trees, there is a one-to-one correspondence between the B_i and the (inner/non-leaf) merge-tree nodes. [1][3]

Merge costs: $M(v)$ is the total length of the resulting merge corresponding to the internal node v . The total merge cost of merge tree T is then: $M(T) = \sum_{v \in T} M(v)$. Given that each node can be attributed to the merge between two runs, we can also calculate the merge costs by analyzing how many times elements are counted: every element in run R_i is counted exactly d_i times in $\sum_v M(v)$, so:

$$M(T) = \sum_{i=0}^{r-1} d_i \cdot L_i \quad (1)$$

[1][3]

2.2 Powersort definitions:

These definitions are related to the Powersort algorithm.

Node Power: Let $\alpha_0, \dots, \alpha_m, \sum \alpha_j = 1$ be leaf probabilities. For $1 \leq j \leq m$, let j be the internal node separating the $(j - 1)$ st and j th leaf. The power of (the split at) node j is:

$$P_j = \min\{\ell \in \mathbb{N} : \lfloor \frac{a}{2^{-\ell}} \rfloor < \lfloor \frac{b}{2^{-\ell}} \rfloor\}, \text{ where } a = \sum_{i=0}^{j-1} \alpha_i - \frac{1}{2} \alpha_{j-1}, b = \sum_{i=0}^{j-1} \alpha_i + \frac{1}{2} \alpha_j. \quad (2)$$

(P_j is the index of the first bit where the (binary) fractional parts of a and b differ.) [3]

Powersort: Powersort is a mergesort algorithm that decides when two runs are ready to be merged through the following merge policy: by simulating a nearly optimal binary search tree through Mehlhorn's second method [2], the algorithm compares the node power p between runs k and $k+1$ with the node power between runs $k-1$ and k , p' . If $p' > p$, the algorithm merges run k with run k' . The algorithm keeps a stack of runs and a stack of powers, therefore, when a merge happens, it pops both

stacks and keeps on comparing the node power on top of the stack with p , until $p' > p$ is false. Once the algorithm reaches the end of the array, it simply merges all runs left in the stack, which guarantees the entire array will be sorted.

Forgetful K-Stack/C-Bounded Stack: A forgetful k -stack is a stack that 'forgets' its first element if it already has k elements in it and the $(k+1)$ -th is added. This is simply a circular stack in which we overwrite bottom elements when the stack is full.

C-Bounded Powersort: A variation of the Powersort algorithm that uses a Forgetful K -Stack instead of the stack with size $\log n + 1$.

Processing a subtree of height h : Let T be a merge tree, T' one of its subtrees of size h and v the root of T' . Processing T' is executing all the merges necessary to fill node v , given that it has length $M(v)$.

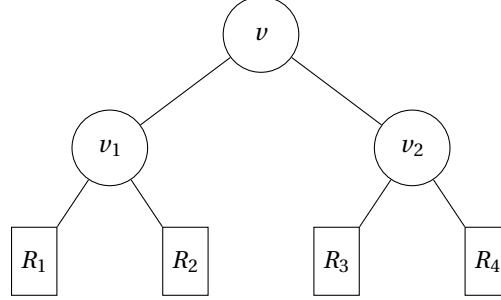
3 Proofs:

Here, we prove that our algorithm is optimal and has the same asymptotic complexity as that of the original Powersort.

Proposition: Let T be a Powersort merge tree of height H . One pass of C-Bounded Powersort is enough to process all subtrees of T with height of $C-1$.

Proof by induction:

Base case - T of height 2:



Let v be the root of T and v_1 and v_2 its two children. Therefore, T has two sub-trees, T' and T'' , both of height one. For the left subtree T' , v_1 is the root and it has only two leaf children: the runs R_1 and R_2 . Given that $h = 1$, a Forgetful 2-Stack will work with 2-Bounded Powersort as follows: R_1 will be scanned and, then, the algorithm will search for the adjacent run, R_2 . After scanning R_2 , the algorithm will calculate the power between R_1 and R_2 , which is equal to the height of v_1 . Given that the stack of powers is empty, there will be no merges and R_1 will be added to the stack. Afterwards, the algorithm will proceed to the right T'' subtree, which is rooted in v_2 . The algorithm will find R_3 and, then, calculate the power between R_2 and R_3 . This will result in the height of v , which is smaller than that of v_1 , our previously calculated power, leading to the merge of the run on top of the stack, R_1 with our current run R_2 , creating run R_{12} . Then, the stack of powers and of runs will be empty

and run R_{12} will be added to the stack of runs whilst the power between R_2 and R_3 will be added to the stack of powers.

After this, our Forgetful 2-Stack will simply contain R_{12} . Then, the algorithm will find run R_4 . Calculating the power between R_3 and R_4 leads to height v_2 , which is bigger than that of v , our previously calculated power, meaning the algorithm will not merge any runs and will add R_3 to the Forgetful 2-Stack. Once we find R_4 , we will have reached the end of the array.

Following the logic of the algorithm, once the scan reaches the end of the array everything that still is in the Forgetful-Stack will be merged with our current run, thus, in this situation, a merge between R_4 and R_3 will happen, creating run R_{34} and both stacks will be popped. Then, a merge between R_{12} and R_{34} will happen, leading to the processing of both subtrees T' and T'' . This concludes our base case.

Hypothesis: Suppose the proposition holds true for all subtrees of height $h \leq k$ in a powersort merge tree.

Inductive Step: Let T be a Powersort merge tree of height H . We will prove that a $(k+2)$ -Bounded Powersort processes all of T 's subtrees that have height of $h = k+1$. Let T' be one of those subtrees and v its root. We will analyze the sub-trees T_ℓ and T_r , each rooted in the left and right children of v , v_ℓ being the left child and v_r the right child. The height of the T_ℓ is, at most, k , therefore, we can apply our induction hypothesis and process T_ℓ , thus, v_ℓ will contain run R_ℓ , which is the result of the merge of all runs in T_ℓ . The processing of this sub-tree will happen once the most-left run of T_r is analyzed by the algorithm.¹ By the end of the processing of T_ℓ , our Forgetful $(k+2)$ -Stack will contain R_ℓ and the most-left run of T_r . Given that, by hypothesis, the processing of T_r requires one pass of a $(k+1)$ -Bounded Powersort² and that our Forgetful $(k+2)$ -Stack has exactly k spaces left, with one space being occupied by the first run of T_r , the algorithm will be able to process T_r . Similarly to what happened during the processing of T_ℓ , the calculation of the power between the right-most run of T_r (which is also the right-most run of T') and the next run, which is the left-most run of another subtree of T with height $k+1$, will result in the height of $v - 1$.⁴ Given that all the powers calculated between runs in T' are greater than $v-1$, the algorithm will merge all the remaining runs in our Forgetful $(k+2)$ stack, leading to the processing of T_r but also to the processing of T' , given that R_ℓ will still be on the stack alongside all runs from T_r .⁵ Therefore, if the $(k+2)$ -Bounded Powersort was able to process a generic T' subtree of T with height $k+1$, it is fair to assume

¹ Once the power between the most-right run of T_ℓ with most-left run of T_r is calculated, it will result in the height of v , which is smaller than the height of v_ℓ , meaning the algorithm will merge everything left in our Forgetful $k+2$ -Stack, for all powers calculated between runs in T_ℓ is greater than v_ℓ

² Remember that T_ℓ and T_r have, at max, height of k

³ Requiring a $(k+1)$ -Bounded Powersort means the processing needs $k+1$ free spaces in the Forgetful C-Stack, not a specific tuning of the algorithm.

⁴ This is due to the fact that both runs are in adjacent subtrees: v is the left child of a node v_p and there is a node v' which is the right child of the same v_p , the first common ancestor between v and v' is v_p , therefore, the first common ancestor between runs in each subtree (the subtree rooted in v and the subtree rooted in v') is v_p

⁵ In reality, the last run of T_r will not be in the stack: it will be the 'current run' in the algorithm whose node power with the consecutive run will result in $v-1$. Merges happen between the current run and the run on top of the stack.

that it will be able to further process all the remaining subtrees of same height by following the same logic as the one described for T' . $\square$

Proposition: It takes C-Bounded Powersort at most $\mathcal{O}(n \cdot \log n)$ element scans to order an array of size n .

Proof. Each pass through the array scans element i , with $0 \leq i \leq n$, at least once. Given that the algorithm will do $\frac{\log n}{c-1}$ passes⁶, i will be read at least $\frac{\log n}{c-1}$ times. Once the entire array is ordered, i will also have been read at least twice for each merge it was present in, i.e, $2 \cdot d_i$, with d_i being equal to the height of T . Therefore, the total number of scans is:

$$Scans \leq \sum_{i=0}^n 2 \cdot \log n + n \cdot \frac{\log n}{c-1} \quad (3)$$

$$Scans \leq n \log n \cdot (2 + \frac{1}{c-1}) \quad (4)$$

$$Scans \leq \mathcal{O}(n \cdot \log n) \quad (5)$$

$\square$

⁶The height of a Powersort merge tree T is $\log n$ and the C-Bounded Powersort algorithm starts by processing subtrees of T that have height less than or equal to $c-1$. This results in a smaller tree T' of height $\log n - (c-1)$. A second pass will, once again, process the subtrees of T' that have height less than or equal to $c-1$, resulting in a smaller tree T'' . Therefore, if k is the amount of trees created by this process until the last tree is of height less than or equal to $c-1$, meaning it will only require one last pass of the algorithm to be processed, $k \cdot (c-1) \leq \log n$

4 Pseudocodes:

TO-DOs:

1. Os comentários para cada algoritmo precisam estar de acordo com as definições usadas no começo do artigo e precisam ser revisados para fazerem sentido, alguns ainda estão bem abstratos e/ou não descrevem corretamente os algoritmos.
2. Formatação geral dos algoritmos para que eles fiquem bem posicionados nas páginas, além de deixar eles mais 'bonitos', não sei qual o formato de apresentação que o professor prefere.

Introduction: In this section, we introduce the original pseudocodes for the Powersort and Node Power algorithms as well as our modified versions.

4.1 Powersorts:

4.1.1 Standard Powersort

Commentary on Wild and Munro's version: A single pass algorithm that sorts the entire array.

Algorithm 1: Powersort($A[1..n]$)

```
1:  $X :=$  stack of runs                                //capacity $\lceil \lg n(n) \rceil + 1$ 
2:  $P :=$  stack of powers                                //capacity  $\lceil \lg n(n) \rceil + 1$ 
3:  $s_1 \leftarrow 1; e_1 \leftarrow \text{ExtendRunRight}(A[1], n)$ 
4: while  $e_1 < n$  do
5:    $s_2 \leftarrow e_1 + 1; e_2 \leftarrow \text{ExtendRunRight}(A[s_2], n)$     // $A[s_2..e_2]$  next run
6:    $p \leftarrow \text{NodePower}(s_1, e_1, s_2, e_2, n)$     // $P_j$  for node  $j$  between  $A[s_1..e_1]$ 
   and  $A[s_2..e_2]$ 
7:   while  $P_{top}() > p$  do
8:     //previous merge deeper in tree than current
9:      $P_{pop}()$     //→ merge and replace run  $A[s_1..e_1]$  by result
10:     $(s_1, e_1) \leftarrow \text{Merge}(X.pop(), A[s_1..e_1]);$ 
11:     $X.push(A[s_1, e_1])$ 
12:     $P.push(p)$ 
13:     $s_1 \leftarrow s_2; e_1 \leftarrow e_2$     //Now  $A[s_1...e_1]$  is the rightmost run
14: while  $\neg X.empty()$  do
15:    $(s_1, e_1) \leftarrow \text{Merge}(X.pop(), A[s_1..e_1]);$ 
```

4.1.2 First variation of C-Bounded Powersort (only goes right)

Commentary on our first version: This algorithm performs multiple passes on the array, processing all subtrees of height $C-1$ with each pass. We remind the reader of how Powersort works: It is implicitly a complete binary tree of size $\log n$. Therefore, the max amount of passes needed to sort the array is:

$$\text{N}^\circ \text{ of scans} \leq \frac{\log n}{C-1} \quad (6)$$

Algorithm 2: First_C-Bounded_Powersort($A[1..n]$)

```

1:  $X :=$  C-bounded stack of runs                                //capacity C
2:  $P :=$  C-bounded stack of powers                              //capacity C
3:  $s_1 \leftarrow 1; e_1 \leftarrow \text{ExtendRunRight}(A[1], n)$ 
4: while  $e_1 < n$  do
5:   //The while above can be tweaked to use max amount of passes
6:   while  $e_1 < n$  do
7:     //We do a pass covering the entire array
8:      $s_2 \leftarrow e_1 + 1; e_2 \leftarrow \text{ExtendRunRight}(A[s_2], n)$ 
9:      $p \leftarrow \text{NodePower}(s_1, s_2)$  //Using our node power algorithm
10:    while  $P_{\text{top}}() > p$  do
11:      //We follow the same merge policy as Munro and Wilde
12:       $P_{\text{pop}}()$ 
13:       $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
14:       $X.\text{push}(A[s_1, e_1]); P.\text{push}(p)$ 
15:       $s_1 \leftarrow s_2; e_1 \leftarrow e_2$ 
16:    while  $\neg X.\text{empty}()$  do
17:       $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
18:    //We're using a limited stack, so we need to check for another
    pass
19:     $s_1 \leftarrow 1; e_1 \leftarrow \text{ExtendRunRight}(A[1], n)$  //If  $e_1 = \text{end}$ , the array is
    sorted

```

4.1.3 Second variation of C-Bounded Powersort (goes left and right!)

Commentary on our second version: This works rather similarly to our first version. The main difference is the fact that, once we reach the end of the array, instead of restarting our scan from the beginning, we scan, starting from the end, to the left, until we reach the beginning of the array, and, thus, we repeat the scan-right process. This is slightly better than the first version, given that it takes advantage of runs still being in cache instead of restarting from the beginning.

The code for the entire algorithm is too big to fit in a single page, therefore, we have subdivided it in three parts, as shown below.

Algorithm 3: C-Bounded_Powersort(A[1...n])

```
1:  $X :=$  C-bounded stack of runs and powers
2:  $s_0 :=$  This is a temp variable for the merges
3:  $s_1 \leftarrow 1; e_1 \leftarrow \text{ExtendRunRight}(1, n)$ 
4: while true do
5:   while  $e_1 < n$  do
6:      $s_2 \leftarrow e_1 + 1; e_2 \leftarrow \text{ExtendRunRight}(s_2, n)$ 
7:     //We do not need variable  $s_2$ , but it helps with clarity.
8:      $p \leftarrow \text{NodePower}(s_1, s_2)$  //  $s_2$  is just  $e_1 + 1$ 
9:     while  $X.size > 1$  and  $X.top().power > p$  do
10:       $s_0 \leftarrow X.pop().boundary$ 
11:      Merge(A,  $s_0, s_1, e_1$ )
12:       $s_1 \leftarrow s_0$ 
13:      X.push( $s_1, p$ )
14:       $s_1 \leftarrow s_2; e_1 \leftarrow e_2$ 
15:   while  $X.size > 1$  do
16:      $s_0 \leftarrow X.pop().boundary$ 
17:     Merge(A,  $s_0, s_1, e_1$ )
18:      $s_1 \leftarrow s_0$ 
19:    $s_1 \leftarrow X.pop().boundary$ 
20:    $s_0 \leftarrow s_1; s_1 \leftarrow e_1; e_1 \leftarrow s_0$ 
21:   //The merge of all runs left in the stack results in a
   single run at the end of the array. We use this run as
   the first run in the left pass. This saves on doing a
   call for scanleft to 'switch sides'.
22:   if  $e_1 == 1$  then
23:     break
24:   //If the run resulting from the merges reaches the
   beginning of the array, it is the entire array.
25:   while  $e_1 > 1$  do
26:     //We're scanning from n down to 1!
27:      $s_2 := e_1 - 1; e_2 := \text{ExtendRunLeft}(s_2, 1)$ 
28:      $p := \text{NodePower}(s_1, s_2)$ 
29:     while  $X.size > 1$  and  $X.top() > p$  do
30:        $s_0 \leftarrow X.pop().boundary$ 
31:       Merge(A,  $s_0, s_1, e_1$ )
32:        $s_1 \leftarrow s_0$ 
33:       X.push( $s_1, p$ )
34:        $s_1 \leftarrow s_2; e_1 \leftarrow e_2$ 
35:   while  $X.size > 1$  do
36:      $s_0 \leftarrow X.pop().boundary$ 
37:     Merge(A,  $s_0, s_1, e_1$ )
38:      $s_1 \leftarrow s_0$ 
39:    $s_1 \leftarrow X.pop().boundary$ 
40:    $s_0 \leftarrow s_1; s_1 \leftarrow e_1; e_1 \leftarrow s_0$ 
41:   //Swapping for to the scan right. The equivalent of what
   we did before scanning left.
42:   if  $e_1 == n$  then
43:     break
```

4.2 Node Power versions:

4.2.1 Standard Node power:

Commentary on Wild and Munro's Node Power: This is so silly and expensive!

Algorithm 4: NodePower(s_1, e_1, s_2, e_2, n)

```
1:  $n_1 := e_1 - s_1 + 1$ ;  $n_2 := e_2 - s_2 + 1$ ;  $\ell := 0$ 
2:  $a := (s_1 + n_1/2 - 1)/n$ ;  $b := (s_2 + n_2/2 - 1)/n$ 
3: while  $\lfloor a \cdot 2^\ell \rfloor = \lfloor b \cdot 2^\ell \rfloor$  do  $\ell := \ell + 1$ 
4: return ( $\ell$ )
```

4.2.2 Our Node power:

Commentary on our Node Power: Ours is cool and epic. It could also be reduced to a single line: `return (__builtin_clz(s1 ^ s2))` (TO-DO: explanation of why it works)

Algorithm 5: NodePower(s_1, s_2)

```
1:  $\text{xor} \leftarrow s_1 \wedge s_2$ 
2:  $\ell \leftarrow \text{\_\_builtin\_clz}(\text{xor})$ 
3: return ( $\ell$ )
```

4.3 The data structure:

Introduction: Wild and Munro's paper shows no specific code for their stack data structure, this is probably due to the fact that they simply use a standard stack of height $\log n + 1$. Thus, we will have no 'counterpart' algorithm to analyze. Our algorithm implements a modified version of a stack which we call:

C-Bounded Stack/Forgetful K-Stack.

4.3.1 C-Bounded Stack:

Commentary on our data structure: When creating this data structure, we had to think of two particularities:

1. This is a stack of limited size.
2. This is a stack that has to consider scans going right and a scans going left.

In order to deal with the limited size, which is the 'forgetful' aspect of the data structure, we simply use circularity: we keep the index of where the next new element will be inserted in the case of a push and increment it. If we reach the end of the array, we assign the free space to be the first element and overwrite it in case of a push. As the stack fills, we also increment a variable for 'size', which corresponds to the amount of elements in the stack. Thus, we keep two variables, one for 'top' and

one for 'size'. Other than that, each element in the stack consists of a run boundary and the power of this run with its predecessor.

Run boundaries: When the algorithm is scanning right, the boundary it keeps is the start of the run found. When the algorithm is scanning left, the boundary it keeps is the end of the run found.

Algorithm 6: C-Bounded Stack(C)

```

1:  $top \leftarrow 0$ 
2:  $size \leftarrow 0$                                 //Starting values for the stack
3:  $MaxSize \leftarrow C$                             //Max stack size in number of elements
4:  $array[MaxSize]$     //The array will store tuples of (index,power)
5: func  $pop(S)$ :
6:   if  $S.size = 0$  then
7:     return underflow
8:    $S.size \leftarrow S.size - 1$ 
9:    $r \leftarrow S.top$ 
10:  if  $S.top = 0$  then
11:     $S.top \leftarrow MaxSize - 1$ ;
12:  else
13:     $S.top \leftarrow S.top - 1$ 
14:  return  $S.array[r]$ 
15: func  $push(S, b, p)$ :
16:   //  $b$  is the boundary of the latest run
17:   //  $p$  is the node power between this and the following run.
18:    $S.top \leftarrow S.top + 1$ 
19:   if  $S.top > MaxSize$  then
20:      $S.top \leftarrow 0$ 
21:   if  $S.size < MaxSize$  then
22:      $S.size \leftarrow S.size + 1$ 
23:    $S.array[S.top].b \leftarrow b$ 
24:    $S.array[S.top].p \leftarrow p$ 
25: func  $TopPower(S)$ :
26:   return  $S.array[S.top].p$ 

```

References

- [1] William Cawley Gelling et al. “Multiway Powersort”. In: *2023 Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX)*. Society for Industrial and Applied Mathematics, Jan. 2023, pp. 190–200. ISBN: 9781611977561. DOI: 10.1137/1.9781611977561.ch16. URL: <http://dx.doi.org/10.1137/1.9781611977561.ch16>.
- [2] K. Mehlhorn. “Nearly optimal binary search trees”. In: *Acta Informatica* 5 (1975), pp. 287–295. DOI: 10.1007/BF00264563.
- [3] J. Ian Munro and Sebastian Wild. “Nearly-Optimal Mergesorts: Fast, Practical Sorting Methods That Optimally Adapt to Existing Runs”. en. In: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018. DOI: 10.4230/LIPICS.ESA.2018.63. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPICS.ESA.2018.63>.